

摘要：

DeepSeek V4技术报告中，一项关键设计引发关注：batch invariance（批次不变性）。为保证同一token在不同batch中输出逐比特一致，V4不惜牺牲GPU利用率与推理速度，舍弃split-KV等常规优化路径，转而自研dual-kernel与DeepGEMM。这一“不计代价”的工程取舍，构成了其复杂系统底层的确定性基石。

DeepSeek V4技术报告，还在被深挖。

常看常新的那种——一个引起关注的新发现是，V4在工程上为了保留核心设计「batch invariance」，都有点不计代价了。

- **Matrix Multiplication.** Traditional cuBLAS library (NVIDIA Corporation, 2024) cannot achieve batch invariance. Therefore, we replace it end-to-end with DeepGEMM (Zhao et al., 2025). Furthermore, for very small batch sizes, conventional implementation usually employs split-k (Osama et al., 2023) techniques to improve performance. Unfortunately, split-k techniques cannot guarantee batch invariance, a pivotal feature in DeepSeek-V4.

怎么说？

DeepSeek V4同时做到了「超长上下文」「复杂后训练/推理管线」「自研高性能kernel栈」这几件很容易打架的事，而背后的关键，正是batch invariance（批次不变性）。

但batch invariance并非没有代价，甚至代价还挺大：GPU利用率、推理速度下降，工程复杂度还更高了……

所以，为什么DeepSeek V4会对这一设计如此执着？我们具体来拆析一下。

什么是batch invariance

先来看看batch invariance的定义：

对于同一个token，无论它在批次里排第几、无论批次多大、无论和谁一起批处理，输出都能保持逐比特完全一致。

论文提到，其核心设计目的，是确保预训练、后训练和推理全流程的可复现性，保证各个环节之间的对齐。

这样做的好处是，首先，能保证线上推理结果稳定。

线上服务会动态batching，同一个用户请求，今天可能和A、B请求拼在一起，明天可能和C、D请求拼在一起。

如果没有batch invariance，同样的提示词就可能因为batch组合不同、底层kernel归约顺序不同等因素，被放大成完全不同的答案。

也就是说，batch invariance能让同一个输入，尽量得到严格一致的输出。

其次，保证预训练、后训练、推理之间的对齐。

DeepSeek V4有预训练、SFT、RL、on-policy distillation、推理服务等多条链路。

这就导致了一个问题：模型行为变化到底是来自数据、RL、蒸馏、量化，还是来自batch shape/ kernel路径变化？

有了batch invariance，工程团队更容易判断，是不是batch组织方式改变了数值结果。

当问题能够被准确定位，异常也就更容易复现，提高了可复现性和可调试性。

另外，**batch invariance**也是复杂上下文系统的底座之一。

V4同时用了长上下文attention、压缩KV、稀疏注意力、MoE、FP4/FP8、Muon、mHC、自研kernel等很多复杂组件。

组件越多，数值不确定性的来源越多。batch invariance相当于给底层执行系统加了一条硬约束：可以优化性能，但不能因为batch变了，就让同一个token的结果变了。

batch invariance还能让后训练更稳定。

RL、蒸馏、长链推理对细微差异很敏感。一点点数值差异，可能改变采样路径；采样路径一变，reward、teacher-student对齐、训练信号都会变。

batch invariance能够减少随机数值扰动，让模型行为更可控。

总结一下，就是batch invariance是DeepSeek V4的底层工程稳定器，可以在在极复杂的长上下文训练、后训练和推理系统里，保证同一输入的数值行为不被batch组织、kernel调度和归约顺序污染，实现可复现、可调试、可对齐、可稳定部署的工程确定性。

牺牲了什么

这样看来，batch invariance确实很重要。

但，就像前面说的，代价不少。

为了batch invariance，V4不能随便使用一些常见性能优化了，比如split-KV、split-K。

在attention里，split-KV会把单条序列的注意力计算分摊到多个SM上，以提高负载均衡和GPU利用率。但这种做法会改变并行归约路径，难以保证同一个token在不同batch组织方式下的逐比特一致性。

在GEMM里，split-K的做法是把矩阵乘法的归约维度K切开并行计算。多路并行求和之后还要再归约，而浮点加法的归约顺序一变，最终结果的bi 就可能不同，因此也和batch invariance存在冲突。

为此，DeepSeek在attention侧提出了**dual-kernel**：为同一个注意力解码任务准备两套计算程序，分别处理“GPU吃得满”和“GPU吃不满”的情况，同时保证两套程序算出来的结果逐比特一致。

矩阵乘法方面，V4在大多数场景中放弃split-K，转而做更受约束的 batch-invariant GEMM。他们用自研**DeepGEMM**替代了通用的cuBLAS。



这些都导致了工程复杂度的明显上升：很多原本可以交给通用库或常规优化策略的工作，都必须由自研kernel和更严格的计算路径来承担。

如此种种，简单总结起来意味着DeepSeek V4在以下几个方面做了牺牲：

- GPU 利用率（波前量化问题）
- 小批量/短序列速度
- 原生算子兼容性
- 部分稀疏加速的自由度

以换取：

- 训练/推理/RL三阶段逐比特可复现
- 长上下文、Agent、RL训练稳定度
- 多机多卡跑出来的结果完全对齐

One More Thing

DeepSeek V4发布了这么些天，但这个技术报告属实是越挖越有。

看点还包括，把10个以上专家教师模型蒸馏成一个学生模型等等。

并且每个问题背后都有坚实的数学解释。

Hugging Face的Transformers负责人Arthur Zucker就感慨：

把数月乃至数年的努力全部免费公开，让任何人都能受益，这是真正的GOAT（Greatest of All Times）。

**Arthur Zucker**  @art_zucker · 19h

Reading @deepseek_ai 's v4 paper.... absolute hats off.

Every problem has a mathematical solution, nothing is left to chance.

I have so much respect for them, putting out months or years of efforts entirely for free, in the open for anyone to benefit. Real goats 🐐

在读 @deepseek_ai 的 v4 论文.....真的要向他们脱帽致敬。

每个问题都有数学解法，世上没有偶然。

我非常敬佩他们，把数月乃至数年的努力全部免费公开，让任何人都能受益。真正的巨匠 🐐

公众号：量子位

本文来源：[量子位](#)

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领

* 体验资格每人限申领一次

扫码
免费领取

VIP会员·7天畅读卡 | 大师课·入门串讲



限免

81 收藏

分享到:

写评论

请发表您的评论

表情 图片

发表评论